

Trabalho Prático 1

Planejamento Urbano da cidade fictícia de Somatório

Thiago Henrique Silva de Almeida

2024014180

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - MG - Brasil

thiagohenriquesilva@ufmg.br

1. Introdução

O presente trabalho aborda um problema de planejamento urbano na cidade fictícia de Somatório. A prefeitura deseja construir novas áreas verdes demolindo ruas existentes, mas sem prejudicar as rotas de trânsito mais eficientes da cidade. O desafio computacional consiste em analisar a malha viária da cidade para identificar ruas com diferentes níveis de importância para os trajetos mais curtos entre dois pontos de interesse: a praça central (região 1) e o parque ecológico (região N). O problema se divide em três partes: determinar a menor distância entre esses dois pontos, identificar todas as ruas que fazem parte de pelo menos um caminho mínimo, e, por fim, encontrar as "ruas críticas", que são aquelas que fazem parte de todos os caminhos mínimos.

2. Modelagem

O problema foi modelado utilizando a teoria dos grafos, conforme solicitado.

2.1. Estruturas de Dados

A cidade de Somatório, com suas N regiões e M ruas, foi representada como um grafo não-direcionado e ponderado.

Os vértices do grafo representam as regiões da cidade, numeradas de 0 a N-1 internamente para corresponder aos índices de um vetor.

As arestas representam as ruas, cada uma conectando dois vértices e possuindo um peso que corresponde ao seu comprimento (distância).

Para a implementação, foi escolhido um vetor de listas de adjacências, utilizando `std::vector<std::list<Rua>>`. Esta estrutura é ideal para representar grafos esparsos, como mapas de cidades, onde uma região não se conecta a todas as outras, otimizando o uso de memória.

2.2. Algoritmos

O algoritmo central empregado para a resolução das três partes foi o Algoritmo de Dijkstra. Ele é projetado para encontrar os caminhos mais curtos a partir de um único vértice de origem em um grafo com pesos de aresta não-negativos, o que se encaixa perfeitamente nos requisitos do problema. Para a parte 3, uma versão modificada do Dijkstra foi implementada para também contar o número de caminhos mínimos distintos, uma informação crucial para a análise.

3. Solução

A solução foi dividida em três métodos, cada um correspondendo a uma parte do problema, todos baseados nos resultados do algoritmo de Dijkstra.

Parte 1: Distância Mínima

A solução para esta parte é a aplicação direta do algoritmo de Dijkstra. O algoritmo é executado uma única vez, partindo do vértice de origem (região 1). Ao final da execução, o valor armazenado no vetor de distâncias para o vértice de destino (região N) corresponde à distância do caminho mínimo.

Parte 2: Ruas em Pelo Menos Um Caminho Mínimo

Para identificar todas as arestas que podem pertencer a um caminho mínimo, a seguinte propriedade foi utilizada: uma aresta (u, v) com peso w está em um caminho mínimo de i para j se, e somente se, $\text{dist}(i, u) + w + \text{dist}(v, j) == \text{dist}(i, j)$.

Esta propriedade é uma consequência direta do Princípio da Otimalidade de Bellman, que afirma que qualquer subcaminho de um caminho mínimo também é um caminho mínimo. A prova mais clara para este princípio é a prova por contradição.

Hipótese: Seja P um caminho mínimo de i para j que passa pela aresta (u, v) . O caminho P pode ser dividido em três partes: o subcaminho de i até u (vamos chamá-lo de P_{iu}), a aresta (u, v) , e o subcaminho de v até j (vamos chamá-lo de P_{vj}). O comprimento total é $\text{comprimento}(P_{iu}) + w(u, v) + \text{comprimento}(P_{vj})$.

Suposição para Contradição: Vamos supor que o subcaminho P_{iu} não seja um caminho mínimo de i para u . Se ele não é o mínimo, então deve existir um outro caminho P'_{iu} de i para u que é mais curto. Ou seja, $\text{comprimento}(P'_{iu}) < \text{comprimento}(P_{iu})$. Se esse caminho P'_{iu} mais curto existe, podemos construir um novo caminho P' de i para j substituindo o trecho P_{iu} por P'_{iu} . O novo caminho seria: $P'_{iu} \rightarrow (u, v) \rightarrow P_{vj}$.

A Contradição: Vamos calcular o comprimento do novo caminho P' : $\text{comprimento}(P') = \text{comprimento}(P'_{iu}) + w(u, v) + \text{comprimento}(P_{vj})$. Como sabemos que $\text{comprimento}(P'_{iu}) < \text{comprimento}(P_{iu})$, então: $\text{comprimento}(P') < \text{comprimento}(P_{iu}) + w(u, v) + \text{comprimento}(P_{vj})$. $\text{comprimento}(P') < \text{comprimento}(P)$.

Isso significa que encontramos um caminho P' de i para j que é mais curto que P . Mas isso é uma contradição, pois nossa hipótese inicial era que P já era um caminho mínimo.

Conclusão: Nossa suposição de que o subcaminho P_{iu} não era mínimo deve ser falsa. Portanto, P_{iu} deve ser um caminho mínimo de i para u . A mesma lógica se aplica para provar que P_{vj} deve ser um caminho mínimo de v para j .

Como os subcaminhos devem ser mínimos, seus comprimentos são, por definição, $\text{dist}(i, u)$ e $\text{dist}(v, j)$. Assim, o comprimento total do caminho mínimo P é $\text{dist}(i, u) + w(u, v) + \text{dist}(v, j)$. E, por definição, o comprimento do caminho mínimo de i para j é $\text{dist}(i, j)$. Portanto, a igualdade $\text{dist}(i, u) + w(u, v) + \text{dist}(v, j) = \text{dist}(i, j)$ é estabelecida.

Primeiramente, executa-se o Dijkstra a partir da origem i para calcular $\text{dist}(i, x)$ para todos os vértices x . Em seguida, executa-se o Dijkstra a partir do destino j para calcular $\text{dist}(j, x)$ para todos os vértices x .

x. Como o grafo é não-direcionado, $\text{dist}(j, v)$ é igual a $\text{dist}(v, j)$. Por fim, o algoritmo itera sobre todas as arestas (u, v) do grafo, verificando se a equação acima é satisfeita. As arestas que cumprem a condição são adicionadas ao conjunto de resultados.

Parte 3: Ruas Críticas

Uma rua é crítica se ela pertence a todos os caminhos mínimos. A solução estende a lógica da Parte 2, incorporando a contagem de caminhos. A condição para uma aresta (u, v) ser crítica é que o número total de caminhos mínimos de i para j seja igual ao número de caminhos mínimos que passam por ela.

A prova para este segundo princípio é feita por um argumento combinatório baseado no princípio da multiplicação.

Definição do Subgrafo de Caminhos Mínimos (SP-DAG): Considere o subgrafo que contém apenas os vértices e arestas que participam de pelo menos um caminho mínimo de i para j . Este subgrafo é um Grafo Acíclico Dirigido (DAG), pois se houvesse um ciclo, poderíamos sempre encontrar um caminho mais curto removendo o ciclo.

Princípio da Multiplicação: Em um DAG, o número total de caminhos de um ponto A para um ponto C que passam por um ponto intermediário B é o (número de caminhos de A para B) multiplicado pelo (número de caminhos de B para C).

Aplicando ao Problema: Queremos contar o número de caminhos mínimos de i para j que passam pela aresta (u, v) . Todo caminho desse tipo é formado por um caminho mínimo de i até u , a aresta (u, v) e um caminho mínimo de v até j .

Contagem: O número de maneiras de escolher a primeira parte é $\text{num_caminhos}(i, u)$. O número de maneiras de escolher a segunda parte é 1 (a própria aresta (u, v)). O número de maneiras de escolher a terceira parte é $\text{num_caminhos}(v, j)$. Pelo princípio da multiplicação, o número total de caminhos mínimos de i para j que passam pela aresta (u, v) é: $\text{caminhos_via_uv} = \text{num_caminhos}(i, u) * 1 * \text{num_caminhos}(v, j) = \text{num_caminhos}(i, u) * \text{num_caminhos}(v, j)$.

Conclusão: Uma aresta (u, v) é crítica se todos os caminhos mínimos de i para j passam por ela. Se todos os caminhos mínimos passam por (u, v) , então o número de caminhos que passam por (u, v) deve ser igual ao número total de caminhos mínimos de i para j . $\text{caminhos_via_uv} = \text{num_caminhos}(i, j)$. Substituindo a expressão que encontramos no passo 4, chegamos à fórmula de verificação: $\text{num_caminhos}(i, u) * \text{num_caminhos}(v, j) = \text{num_caminhos}(i, j)$, se esta equação for verdadeira, a aresta (u, v) é, por definição, crítica.

Executa-se um Dijkstra modificado a partir da origem i , que calcula tanto as distâncias mínimas ($\text{dist}(i, x)$) quanto o número de caminhos mínimos distintos ($\text{num_caminhos}(i, x)$). O mesmo Dijkstra modificado é executado a partir do destino j , obtendo $\text{num_caminhos}(j, x)$. Finalmente, o algoritmo itera sobre todas as arestas (u, v) e verifica duas condições: a aresta deve pertencer a um caminho mínimo (usando a fórmula da Parte 2) e a contagem de caminhos que passam por ela ($\text{num_caminhos}(i, u) * \text{num_caminhos}(j, v)$) deve ser igual ao total de caminhos mínimos de i para j ($\text{num_caminhos}(i, j)$).

4. Análise de Complexidade

A análise de complexidade de tempo e memória, para um grafo com V vértices (N regiões) e E arestas (M ruas), é a seguinte:

4.1. Algoritmo Base (Dijkstra com Fila de Prioridade)

Tempo: $O(E \log V)$. A implementação utiliza uma fila de prioridade (heap binário), onde cada vértice é inserido e removido uma vez ($O(V \log V)$) e cada aresta pode levar a uma atualização na fila ($O(E \log V)$).

Memória: $O(V)$ para os vetores de distâncias e antecessores.

4.2. Parte 1

Tempo: $O(E \log V)$, custo de uma única execução de Dijkstra.

Memória: $O(V+E)$ para armazenar o grafo, mais $O(V)$ para os vetores de Dijkstra, resultando em $O(V+E)$.

4.3. Parte 2

Tempo: $O(E \log V)$. São duas execuções de Dijkstra e uma travessia linear em todas as arestas ($O(V+E)$). O termo dominante é o de Dijkstra.

Memória: $O(V+E)$ para o grafo, mais $O(V)$ para cada um dos dois vetores de distância, resultando em $O(V+E)$.

4.4. Parte 3

Tempo: $O(E \log V)$. A lógica é a mesma da Parte 2 (dois Dijkstras e uma travessia), pois a contagem de caminhos é feita dentro do próprio Dijkstra sem custo assintótico adicional.

Memória: $O(V+E)$ para o grafo, mais $O(V)$ para cada um dos quatro vetores (duas distâncias, duas contagens de caminhos), resultando em $O(V+E)$.

Todas as soluções apresentam complexidade sub-quadrática, atendendo aos requisitos de performance do trabalho.

5. Considerações Finais

A realização deste trabalho prático foi uma experiência de aprendizado extremamente gratificante e desafiadora, que transcendeu a simples implementação de um algoritmo. A jornada de traduzir um problema do mundo real, o planejamento urbano de Somatório, em uma solução computacional concreta e eficiente foi fundamental para conectar a teoria de grafos com a prática da engenharia de software. A satisfação ao final do projeto não reside apenas na obtenção das respostas corretas para os testes, mas no desenvolvimento de uma solução completa, robusta e bem projetada.

Uma das principais dificuldades e, ao mesmo tempo, um dos maiores aprendizados, esteve na modelagem das classes Cidade e Rua. A ideia inicial de encapsular toda a lógica e os dados nos objetos levou a um debate interno sobre o design: Deveriam os objetos Rua armazenar os resultados de uma busca (como a distância da origem ou o número de caminhos), ou deveriam ser stateless? A decisão de adotar a segunda abordagem, onde a Rua armazena apenas suas propriedades intrínsecas (índice, comprimento) e seus métodos recebem o contexto da busca como parâmetro, foi crucial. Isso evitou problemas complexos de gerenciamento de estado, prevenindo bugs de dados obsoletos entre diferentes execuções, e resultou em um design mais limpo, flexível e alinhado ao Princípio da Responsabilidade Única. A refatoração do algoritmo de Dijkstra para uma função auxiliar privada, com

sobrecargas públicas para diferentes necessidades, também foi um passo importante para a criação de uma interface de classe mais clara e reutilizável.

O desenvolvimento foi fortemente pautado pela busca de robustez, impulsionada pelas rígidas exigências de compilação, como o uso da flag `-Werror`. Lidar com os avisos transformados em erros, como `-Wconversion` e `-Wsign-compare`, forçou um entendimento mais profundo sobre os tipos de dados em C++, como a diferença entre `int` e `size_t`, e a adoção de práticas de programação defensiva. A maior lição nesse quesito foi a implementação das verificações de overflow para `long long int`. Perceber que não apenas a soma de distâncias, mas também a multiplicação do número de caminhos poderia estourar o limite do tipo de dado, e implementar soluções seguras (com verificação prévia para adição e usando divisão para a multiplicação), foi um exercício prático de como antecipar e prevenir falhas críticas em casos de teste com entradas muito grandes. O uso de exceções para tratar erros de entrada e de execução também contribuiu para um programa que falha de forma graciosa e informativa.

Ao final, a sensação é de grande satisfação. Superar os desafios, desde a configuração do Makefile até a depuração de bugs lógicos sutis e a implementação de verificações de segurança, transformou este trabalho em muito mais do que um exercício de algoritmos. Foi uma lição prática sobre a importância do bom design, da escrita de código limpo e, acima de tudo, da atenção aos detalhes que transformam um programa que "funciona" em um programa que é verdadeiramente robusto e confiável.

6. Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. (Capítulos sobre Caminhos Mínimos de Fonte Única).

cppreference.com. (2025). C++ reference. Acessado em 02/10/2025. Utilizado para consulta de funcionalidades da STL, como `std::priority_queue`, `std::vector` e `<limits>`.

CP-Algorithms. (s.d.). Dijkstra's algorithm. Finding the number of shortest paths. Acessado em 04/10/2025, de <https://cp-algorithms.com/graph/dijkstra.html#finding-the-number-of-shortest-paths>

AHIRE, Deepak T.; JADHAV, Omkar S. Commentary - Methods to find all the edges on any of the shortest paths between two given nodes of a directed acyclic graph. Rio Journal, 25 maio 2021. Disponível em: <https://riojournal.com/article/68600/>. Acessado em 04/10/2025.

